UNIVERSITY OF PADOVA
DEPARTMENT OF INFORMATION ENGINEERING

Biomedical Wearable Technologies
for Healthcare and Wellbeing

State Management in Flutter

A.Y. 2025-2026

Giacomo Cappon

DEPARTMENT OF
INFORMATION
ENGINEERING
UNIVERSITY OF PADOVA

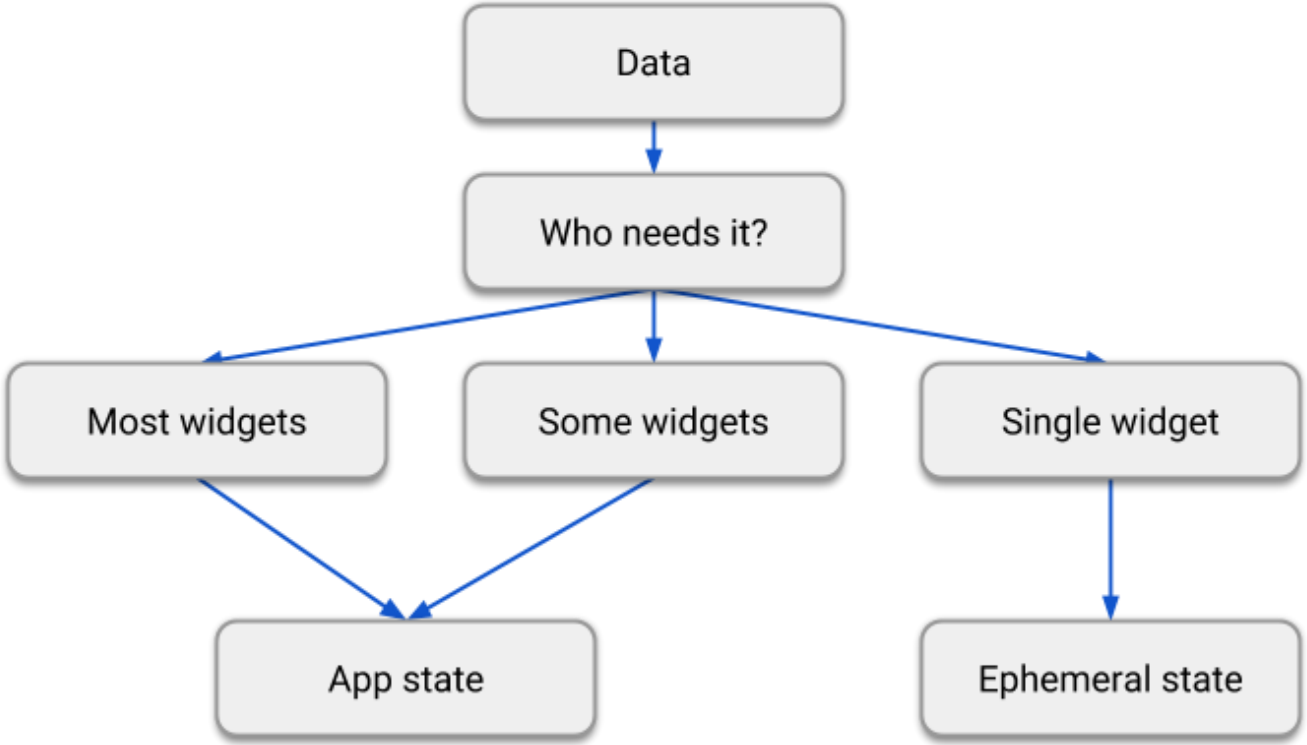
State

- State stands for everything that is necessary to define how the app and its screen behave and look at some point in time:
  - Assets
  - Variables
  - Fonts
  - ...

- Conceptually can be divided in:
  - **Ephemeral state**: sometimes called *local state*, what can be striclty contained in a Widget
  - **App state**: sometimes called *shared state*, things that you want to share across many parts of the app

State

- There is no universal rule to chose if a variable is part of the ephemeral state or the app state. A diagram that can help:



Remember: Flutter is declarative

- Flutter is a declarative framework

- State is changed? Build methods are called and the UI is refreshed.

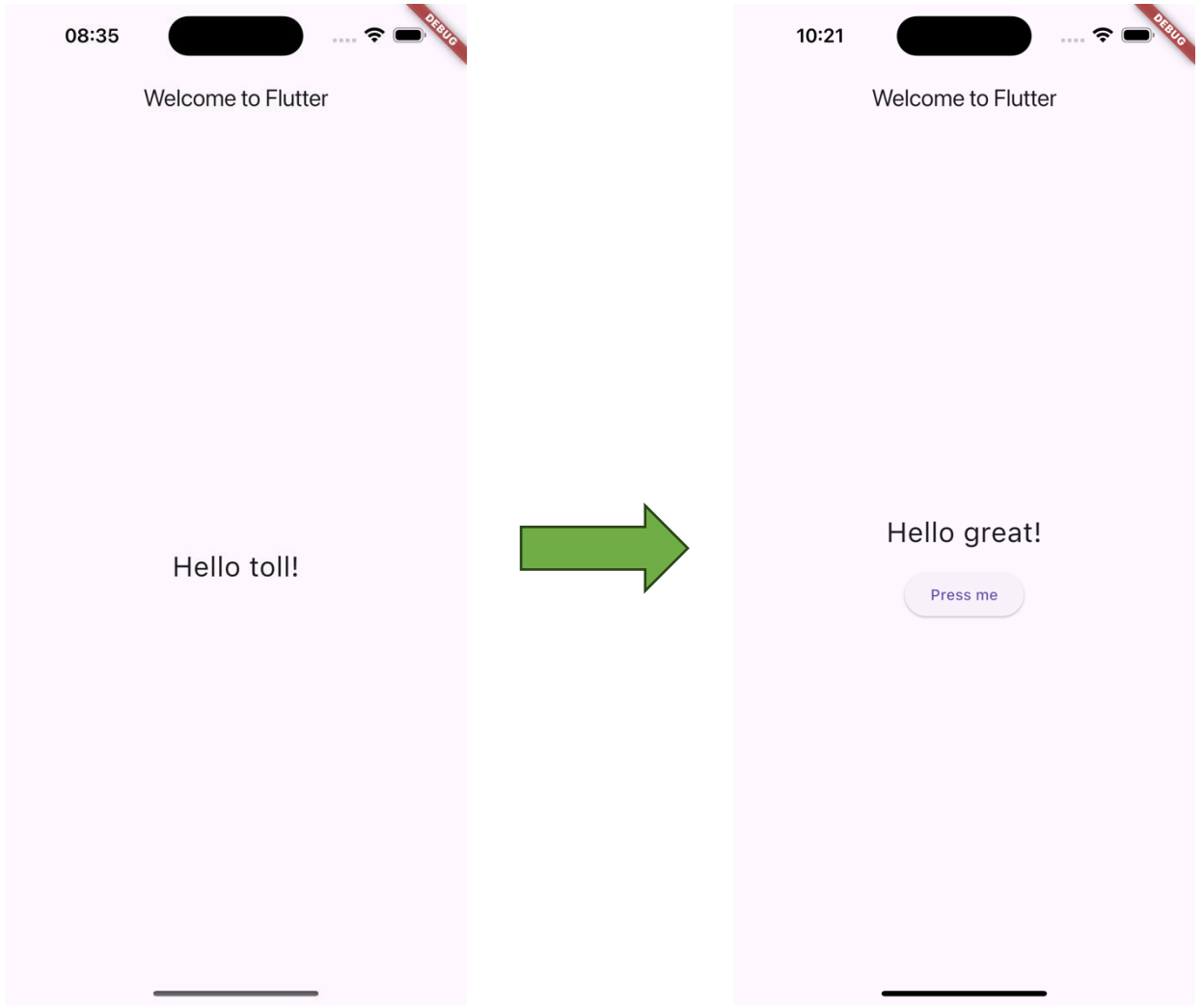
So far…

- **In lesson 5** we built a very simple app that, whenever you open it, a new random ”Hello” message is shown to the user.
  - Code available at: *lesson_05-the_flutter_framework/my_custom_first_app/*

- Question #1: What is “state” here?
- Question #2: How the state is managed? What’s the flow?

So far…

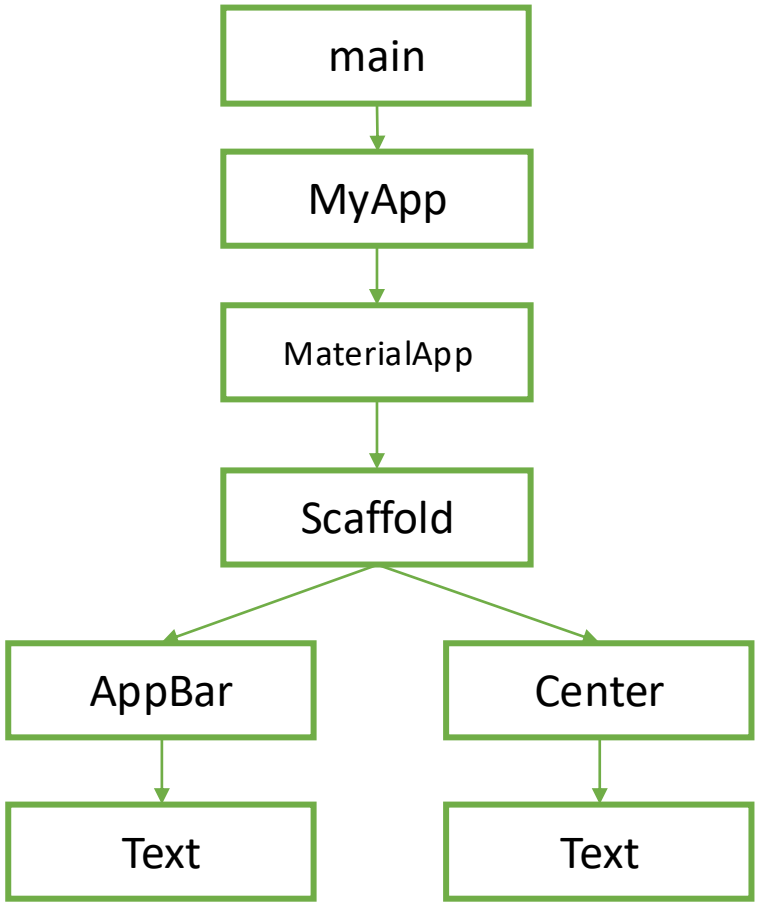
- Let’s expand the app as follows:
- Problem: how to change the UI dynamically?



The Widget Tree

- Remember: Widgets are combined together using a **tree structure**

```
class MyApp extends StatelessWidget {
  MyApp({Key? key}) : super(key: key);


  @override
  Widget build(BuildContext context) {
    final word = WordPair.random().first;

    return MaterialApp(
      title: 'Welcome to Flutter',
      home: Scaffold(
      appBar: AppBar(title: Text('Welcome to Flutter'),),
      body: Center(child: Text('Hello $word!', style: TextStyle(fontSize: 26) ,),),),
    );
  }//build
}//MyApp
```



The Widget Tree

- Remember: Widgets are combined together using a **tree structure**

```
class MyApp extends StatelessWidget {
  MyApp({Key? key}) : super(key: key);


  @override
  Widget build(BuildContext context) {
    final word = WordPair.random().first;

    return MaterialApp(
      title: 'Welcome to Flutter',
      home: Scaffold(
      appBar: AppBar(title: Text('Welcome to Flutter'),),
      body: Center(child: Text('Hello $word!', style: TextStyle(fontSize: 26) ,),),),
    );
  }//build
}//MyApp
```

MyApp is not just a Widget, it is a StatelessWidget

Stateless vs. Stateful widgets

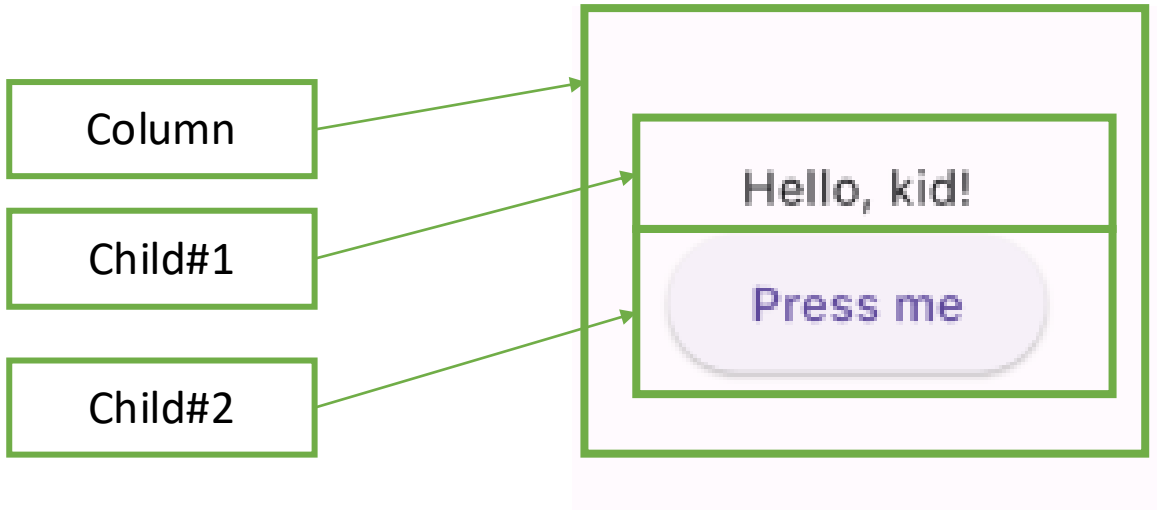
- There are **two main types of Widgets:**
  - **StatelessWidget**s:Widgets that always build the same way given a particular configuration and ambient state. So, they never re-build while they are displayed to the user (their lifetime).
  - **StatefulWidget**s: Widgets that can build differently several times over their lifetime.
- You can think about StatelessWidget as a sort of constant and StatefulWidget as a variable.

Changing the UI

- Let’s start by simply changing the UI

- Problems:

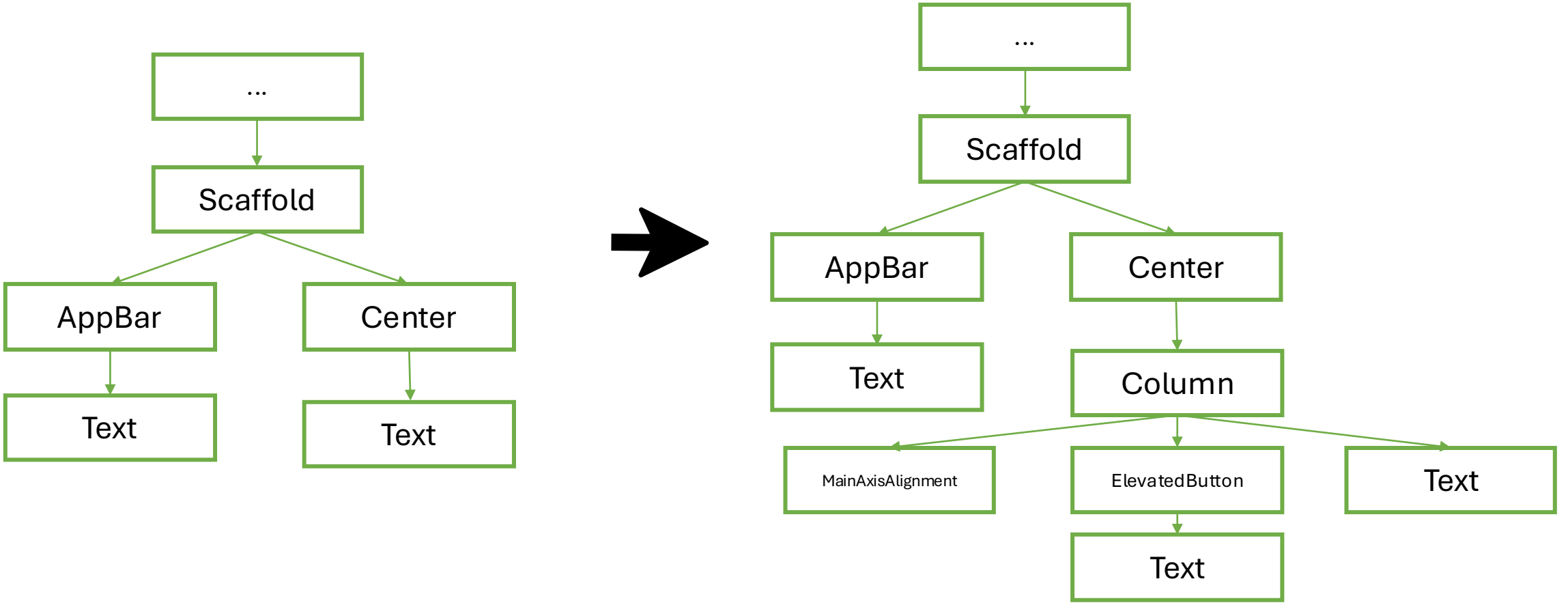
  1. How to add a button
  2. How to put it there



- We know how! We can use the Column Widget

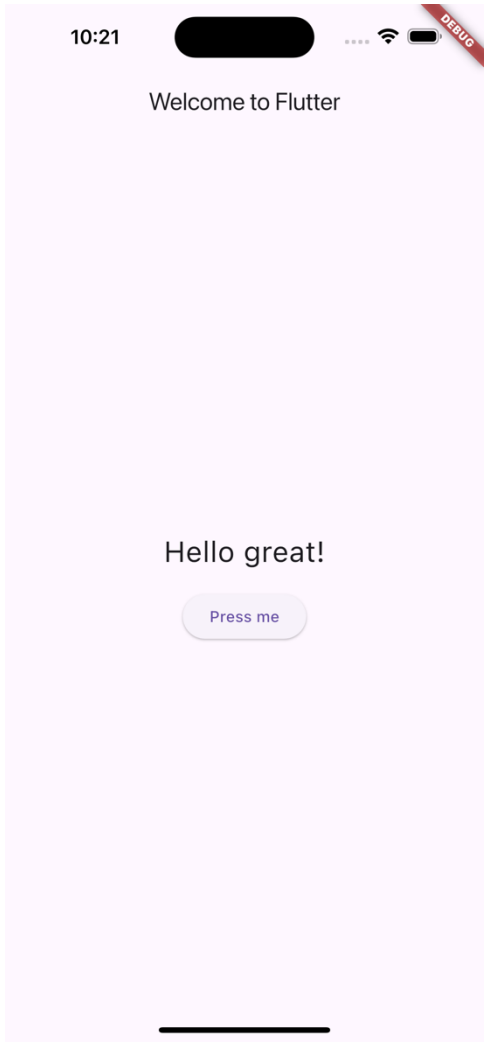
Different UI, different tree

- How the widget tree changed?



Changing the UI

- Problem: how to change the UI dynamically?
- In other words: how to change the **app state** without reloading or restarting everything
- We need a **StatefulWidget**



StatefulWidget

- As we mentioned before, **StatefulWidget** maintain state that might change during the lifetime of the widget.

- Implementing a stateful widget requires at least two classes:

1. A **StatefulWidget class** that creates an instance of the Widget itself

2. A **State class:** a class that manages the state of the **StatefulWidget**

The boilerplate code of a StatefulWidget

```
class ClassName extends StatefulWidget{
  ClassName({Key? key}) : super(key: key);

  @override
  _ClassNameState createState() => _ClassNameState();
}//RandomHello

class _ClassNameState extends State<ClassName>{

  @override
  Widget build(BuildContext buildContext){
    //return some widget
  }//build

}//_ClassNameState
```

Our new code

- Now let's refactor the MyApp code to be more modular

- We can put all the "scaffold-related" code in a new monolithic Widget

```
class HomePage extends StatefulWidget {
  HomePage({Key? key}) : super(key: key);

  @override
  State<HomePage> createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {
  @override
  Widget build(BuildContext context) {
    final word = WordPair.random().first;

    return Scaffold(
      appBar: AppBar(title: Text('Welcome to Flutter'),),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text('Hello $word!', style: TextStyle(fontSize: 26)),
            SizedBox(height: 16),
            ElevatedButton(
                                                child: Text('Press me'),
            ),
          ],
        ),
      ),
    );
  }}//HomePage
```

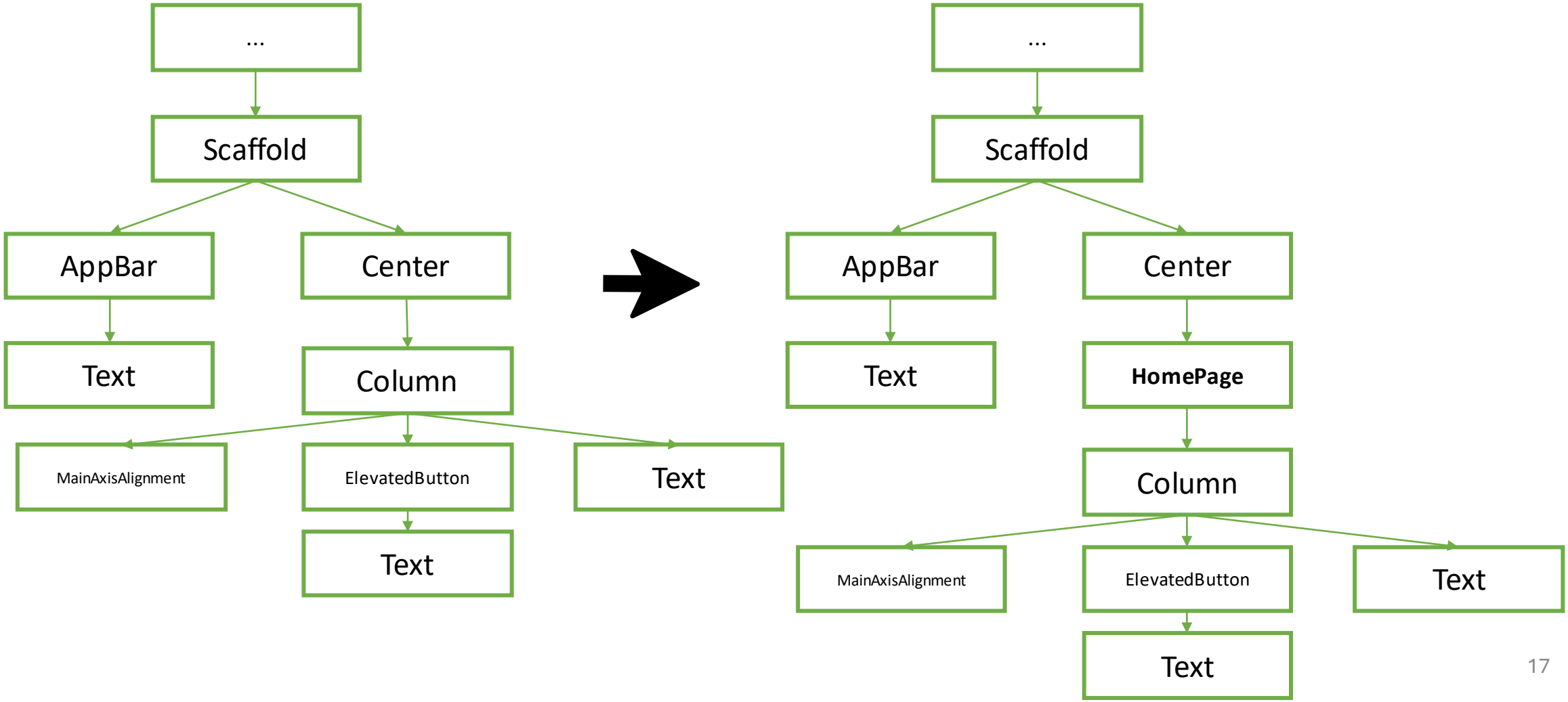
Our new code

- And refactor MyApp like that

```
void main() {
  runApp(MyApp());
}//main

class MyApp extends StatelessWidget {
  MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Welcome to Flutter',
      home: HomePage(),
    );
  }}//MyApp
```

Same UI, different tree

- The UI should look like the same as before, but we have a new widget tree



void initState(){}

- Let's do some changes to HomePage to make it more **stateful**

```
class _HomePageState extends State<HomePage>{

  String? _word;

  @override
  void initState() {
    _word = WordPair.random().first;
    super.initState();
  }//initState

...
```

_word will represent the state of the Widget

initState is a special method that is called the first time the Widget is created. It is used (as its name suggest) to initialize the state of the Widget itself.

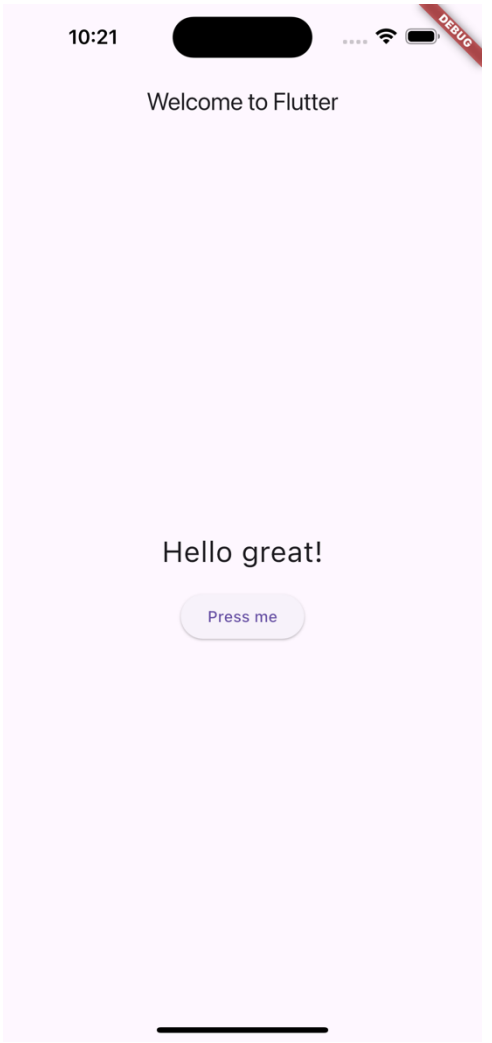
setState((){})

- We are ready to implement the function to provide to onPressed

```
...
@override
  Widget build(BuildContext context) {

    return Scaffold(
      appBar: AppBar(title: Text('Welcome to Flutter'),),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text('Hello $_word!', style: TextStyle(fontSize: 26)),
            SizedBox(height: 16),
            ElevatedButton(
              onPressed: () {
                setState(() {
                  _word = WordPair.random().first;
                });
              },
              child: Text('Press me'),
            ),
          ],
        ),
      ),
    );
...
```

setState is a special method that requires a function as input. setState notifies the Flutter framework that the state might be changed causing to delete and rebuild the widget itself.

We did it!

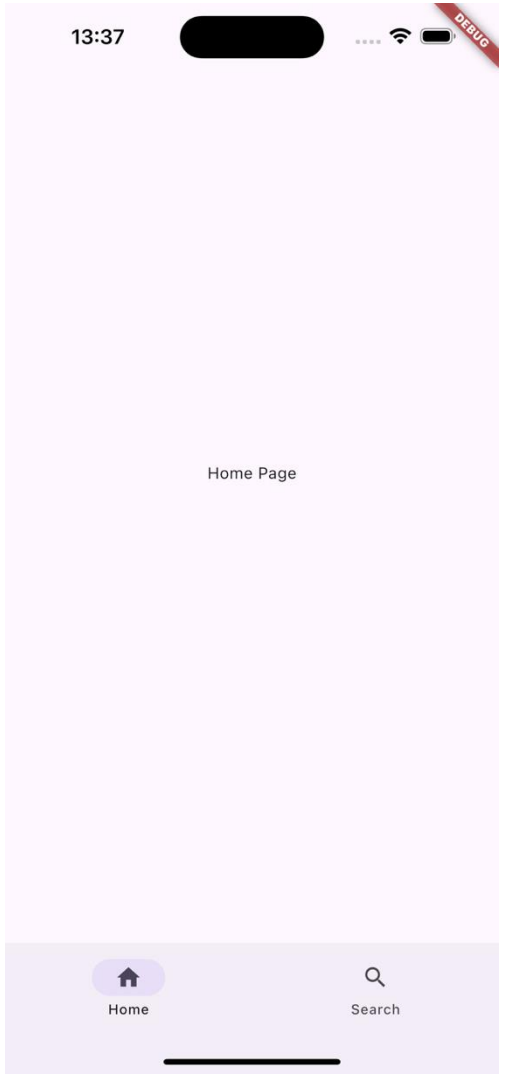


Full example in lesson_11-the_flutter_framework/my_custom_first_app_with_steroids/

A specific use case - NavigationBar

- A particular use case of setState() is the following:

```
...
class HomeScreen extends StatefulWidget {
 const HomeScreen({super.key});

 @override
 State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
 int selectedIndex = 0;

 // List of pages
 final List<Widget> pages = [
  Center(child: Text("Home Page")),
  Center(child: Text("Search Page")),
 ];

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   body: pages[selectedIndex],

   bottomNavigationBar: NavigationBar(
    selectedIndex: selectedIndex,

    // When user taps an item
    onDestinationSelected: (int index) {
     setState(() {
      selectedIndex = index;
     });
    },

    destinations: [
     NavigationDestination(
      icon: Icon(Icons.home_outlined),
      selectedIcon: Icon(Icons.home),
      label: "Home",
     ),
     NavigationDestination(
      icon: Icon(Icons.search_outlined),
      selectedIcon: Icon(Icons.search),
      label: "Search",
     ),
    ],
   ),
  );
 }
}
...
```



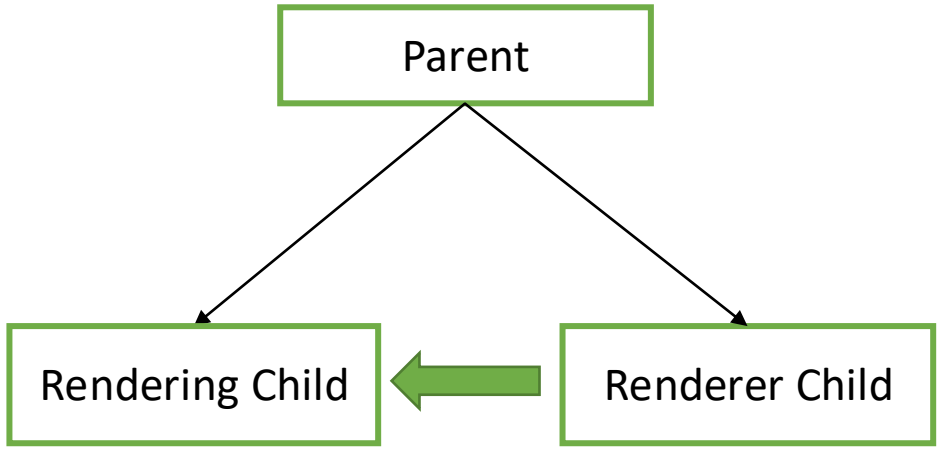
Full example in lesson_11-the_flutter_framework/navigation_bar_example/

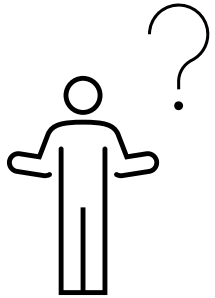
The limitation of setState()

- Why we cannot fully depend on setState() ?

- In a typical widget tree, when two children of a parent are not in the same class as of the parent, rendering one child from another becomes impossible until you implement very messy logic.

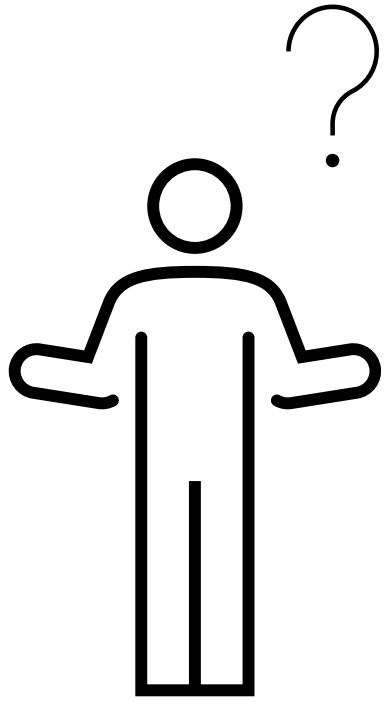
- We DO NOT want that





So, how to manage state?

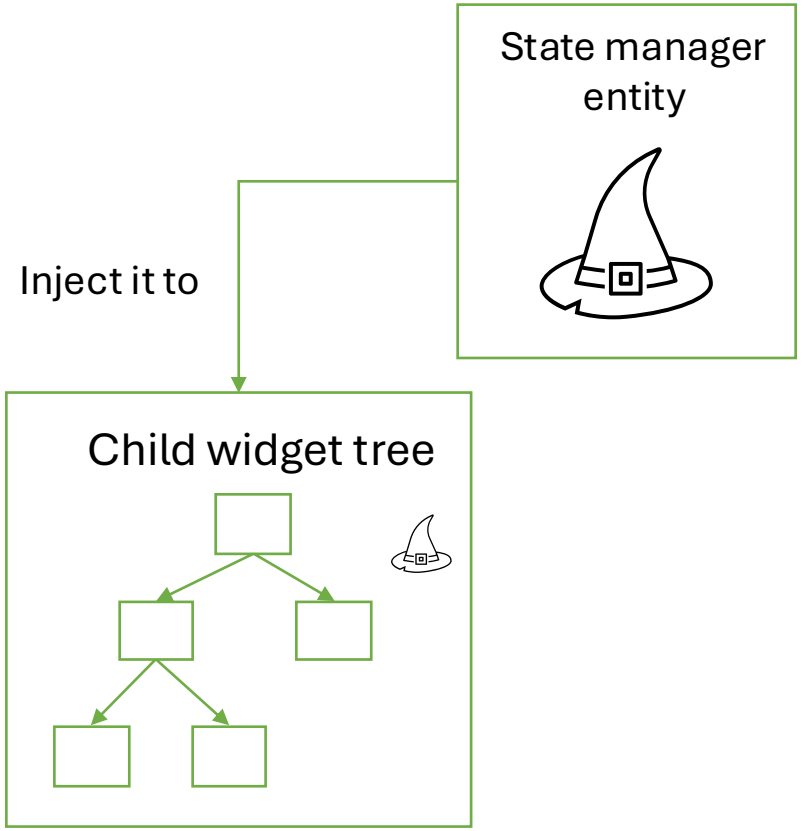
- There is no such a thing as a "*universal way to manage state*"



- There are a lot of possible approaches:
  - Provider
  - Riverpod
  - Redux
  - BLoC
  - …
- Every approach has its PROs and CONs. So?
- Here, we will discuss **Provider**, the recommended approach by the Flutter community.

Provider core idea

- Provider wants to provide!
- The core idea is to provide the entity (one or more classes) in charge of maintaining the state down through the widget tree
- Each child will be able to access to the entity and react to state changes.

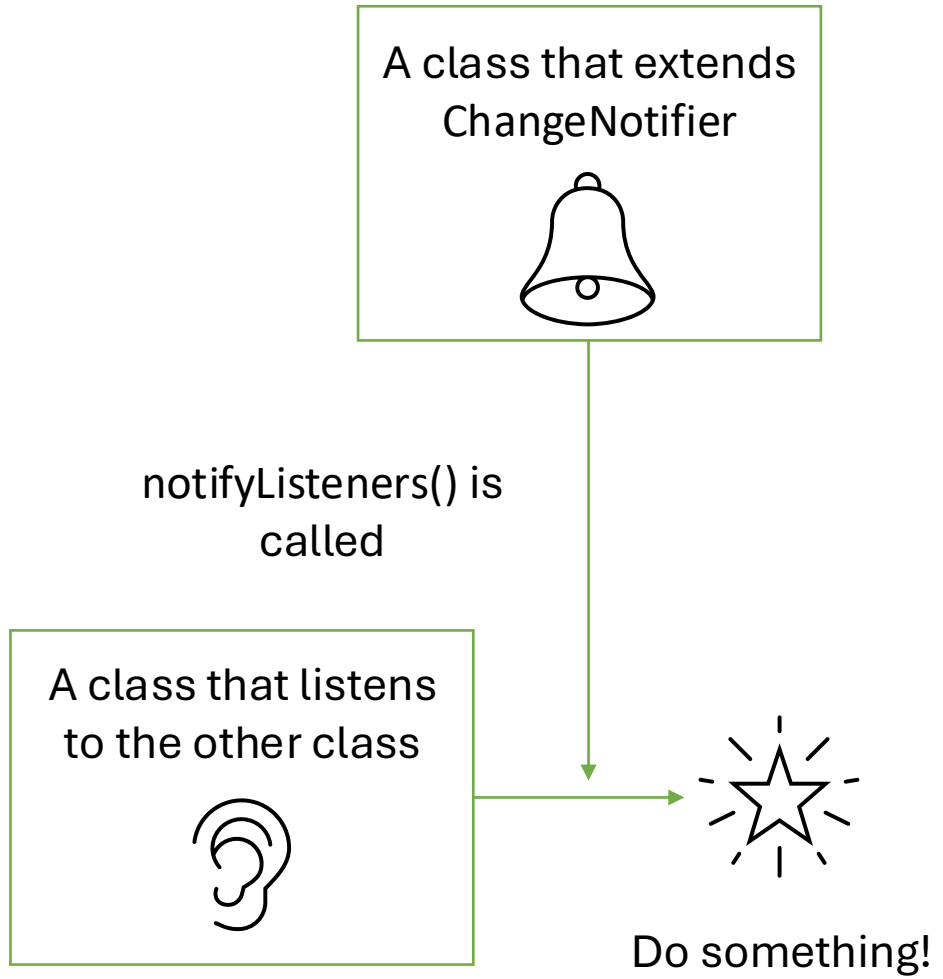


Provider classes

- Provider implement a set of classes, here's the most important:
  - ChangeNotifier
  - ChangeNotifierProvider
  - Consumer
  - MultiProvider
  - …

Provider classes: ChangeNotifier

- ChangeNotifier is a class that can notify **listeners** of any changes in the state.
- You usually extend the ChangeNotifier for models so you can send notifications when your model changes.
- When something in the model changes, you call **notifyListeners**() and whoever is listening can use the newly changed model.



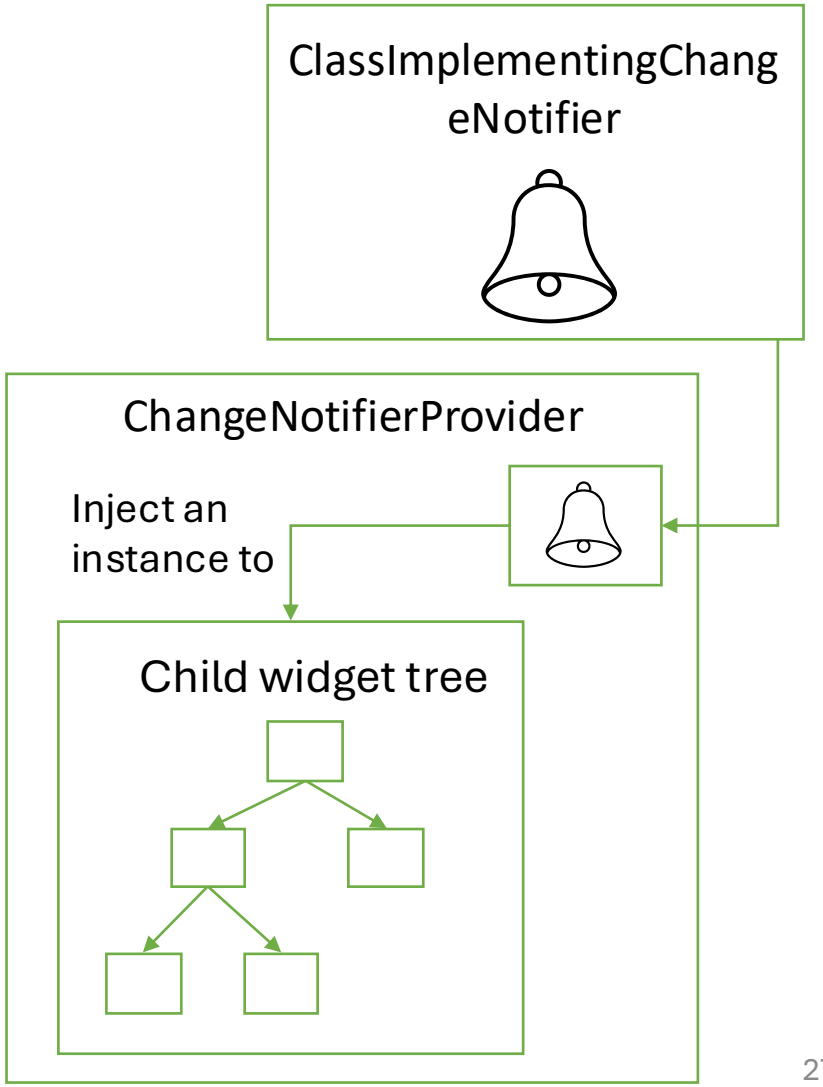
Provider classes: ChangeNotifierProvider

- ChangeNotifierProvider is a class that wraps a class that implements ChangeNotifier and **provide it** to its descendants.

- Now the widget tree can access to it (and use it!)

- Synthax:

```
ChangeNotifierProvider(
        create: (context) =>
        ClassImplementingChangeNotifier(),
        child: <widget_tree>,
);
```

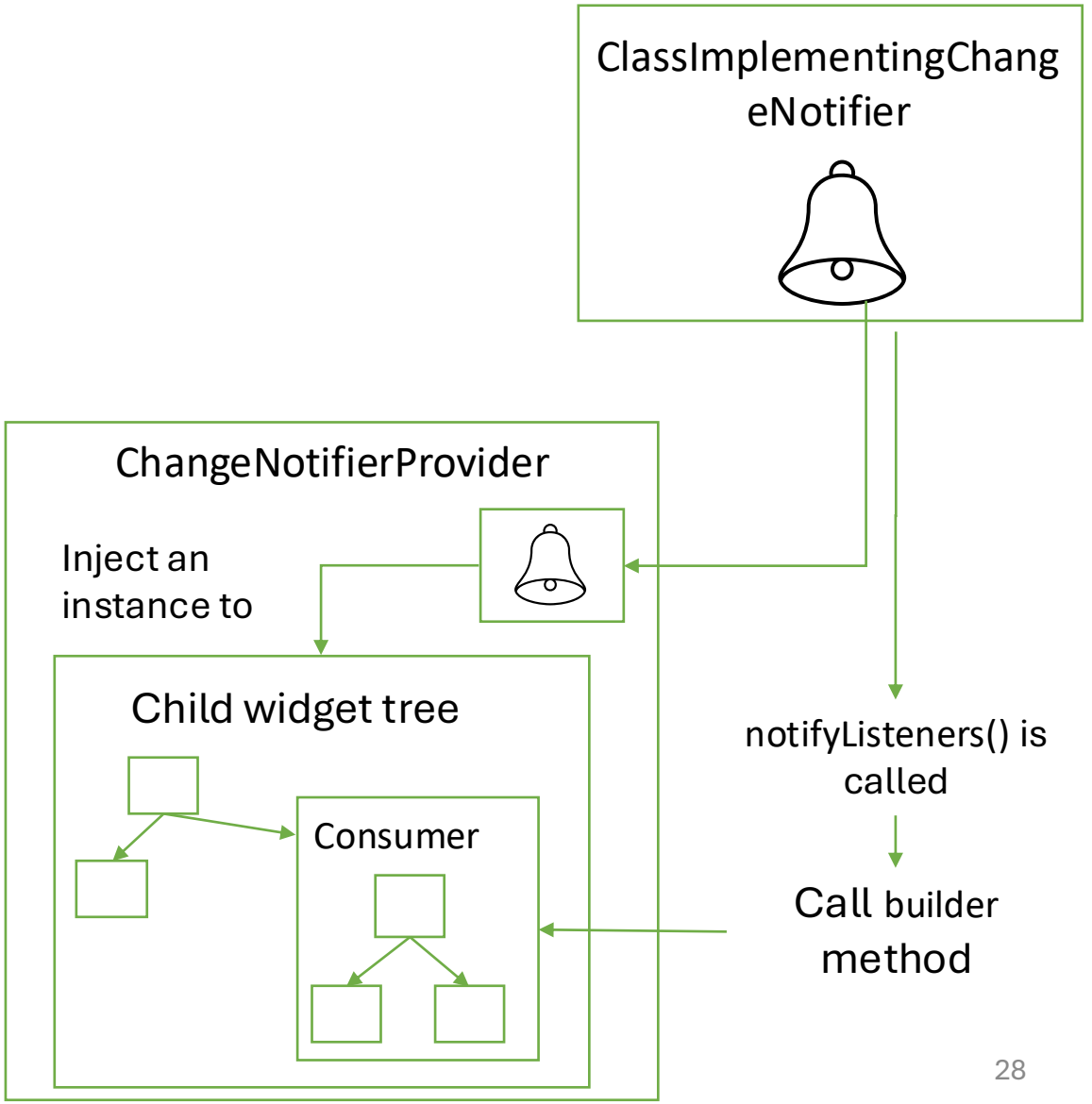


Provider classes: Consumer

- Consumer is a widget that listens for changes in a class that implements ChangeNotifier, then rebuilds the widget tree below itself when it finds any.

- Whenever notifyListeners() is called, the `Consumer's builder` function is called.
Synthax:

```
return
Consumer<ClassImplementingChangeNotifier>(
 builder: (context, widget, child) {
  return Text(<use widget>);
  },
);
```



Provider classes: MultiProvider

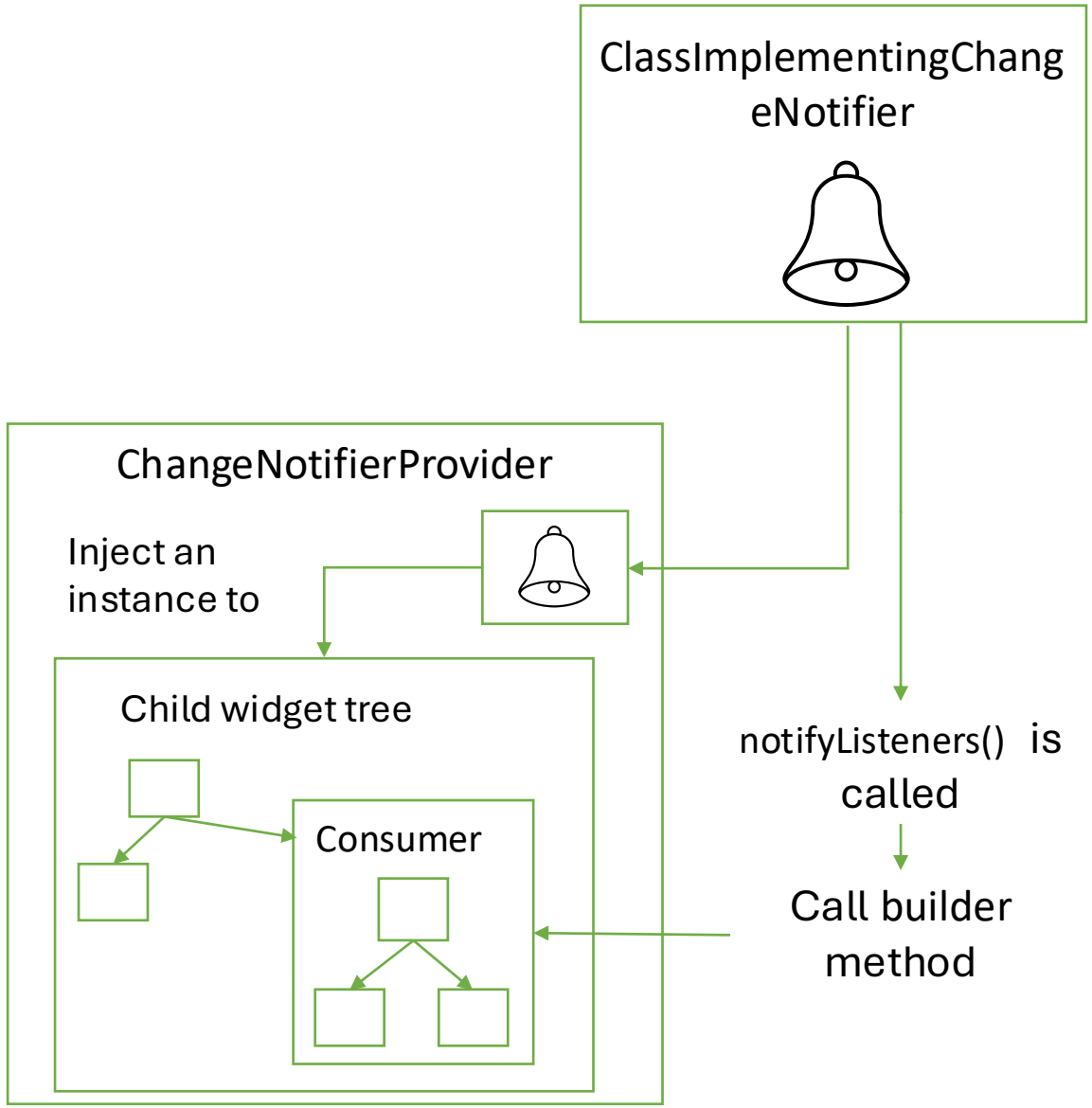
- What if you need to inject more than one provider through the widget tree? Use MultiProvider .

- Synthax example:

List of providers.

```
MultiProvider(
  providers: [
        Provider<ChangeNotifier1>(create: (_) => ChangeNotifier1()),
        Provider<ChangeNotifier2>(create: (_) => ChangeNotifier2()),
        ...
  ],
  child: <widget_tree>,
);
```

Wrapping up: Provider (core) flow



Resources

- State management
  - https://docs.flutter.dev/development/data-and-backend/state-mgmt/intro
- Making sense of all those Flutter Providers
  - https://medium.com/flutter-community/making-sense-all-of-those-flutter-providers-e842e18f45dd
- List of state management approaches
  - https://docs.flutter.dev/development/data-and-backend/state-mgmt/options